

UNIVERSITÀ DEGLI STUDI DI PERUGIA
Dipartimento di Matematica e Informatica

CORSO DI LAUREA TRIENNALE IN INFORMATICA



Tesi di Laurea Triennale

Visualizzazione dati per coaching report

Laureando:

Massimiliano Baglioni

Relatore:

Prof. Osvaldo Gervasi

Dott. Damiano Perri

Anno Accademico 2021–2022

Indice

Introduzione	iv
1 LibreEOL e importanza dei report	2
1.1 Importanza dei report	4
2 Tecnologie Utilizzate	7
2.1 HTML	8
2.2 CSS	9
2.3 JavaScript Chart.js e jQuery	10
2.4 PHP	12
2.5 MySQL	14
2.6 AJAX	14
2.7 JSON	15
2.8 XML	15
3 Implementazione dei report	17
3.1 CoachingReport View	17
3.2 Controller e Azioni	20
3.3 Funzioni Exam Controller	22
3.4 Funzioni nello specifico	23

3.4.1	Risposte di uno studente	23
3.4.2	Funzione per ottenere la media da un insieme di domande	24
3.4.3	Funzione per ottenere il numero di domande giuste per categoria	25
3.5	Query e classe sqlDB.php	26
3.6	Implementazione Grafici	35
3.7	Descrizione contenuto grafici specifici per ogni studente . . .	37
3.8	Contenuto grafici globali	39
3.9	Codice grafici	41
3.10	Traduzione	44
3.11	File JavaScript	45
3.12	CSS	50
3.13	Migrazione codice da locale a web server del dipartimento . .	52
4	Processo di sviluppo	56
4.1	Prima suddivisione del codice	56
4.2	Secondo refactoring	57
	Conclusioni	60
	Bibliografia	61
A	Codice	63

Elenco delle figure

3.1	Vista degli studenti che hanno sostenuto l'esame. Per accedere ai report è presente un bottone in basso a destra.	18
3.2	I due dropdown presenti nella pagina.	20
3.3	Grafico EER	29
3.4	Schermata Grafici Studente	37
3.5	Schermata Grafici Globali	43
3.6	Schermata Grafici Globali	44

Listati di codice

3.1	Funzione accessRules	21
3.2	Funzione actionStudentanswers	23
3.3	Funzione actionAveragefromset	24
3.4	Funzione actionAverageFromset	25
3.5	Esempio codice Query	33
3.6	Esempio di Data per grafico in chartJs	41
3.7	Esempio di options per grafico in chartJs	41
3.8	Funzione per creare grafico in chartsJs	42
3.9	Esempio Traduzione	45
3.10	Esempio funzione con chiamata AJAX	46
3.11	Codice funzione getAJaxCall	46
3.12	Esempio funzione con chiamata AJAX language	49
3.13	Inizializzazione oggetto globalExamInfo	49
3.14	Esempio funzione con chiamata AJAX	50
3.15	Esempio funzione con chiamata AJAX language	53
4.1	Esempio funzione con chiamata ajax	57
4.2	Creazione di tutti i grafici studenti	59
4.3	Ciclo for per utilizzo dell'oggetto options	59

Introduzione

Il progetto svolto come tesi di laurea prevedeva l'implementazione di strumenti per effettuare reportistica riguardo esami svolti sulla piattaforma online **LibreEOL**. Per ottenere il sistema di reportistica sono stati utilizzati dei grafici interattivi che mostrano all'utente diverse prospettive riguardo i risultati dei singoli utenti e del collettivo. L'implementazione richiesta viene effettuata sulla base di un software già esistente da anni e quindi è stato necessario seguirne la struttura e le forme per svolgere al meglio il lavoro.

Nel presente elaborato, che descrive nel dettaglio il progetto, saranno indicate le **tecnologie** usate, i **linguaggi**, le **strutture** e più nello specifico l'**implementazione** vera e propria del progetto. Verranno anche descritti in linea generale gli strumenti utilizzati, le idee e i procedimenti, con attenzione anche a delle porzioni di codice utilizzate all'interno dell'implementazione. Non sarà riportato il codice per intero, ma verranno mostrati esempi di strutture frequenti, verranno spiegate le loro caratteristiche e verrà descritto come queste strutture sono state pensate per svolgere i loro compiti al meglio.

Dove incontrate, verranno anche descritte le **difficoltà** avute e le soluzioni applicate per la risoluzione di queste ultime. In linea generale si descriverà anche l'utilità della reportistica sia in ambiente scolastico che non, verrà

accennata la storia del software **LibreEOL** e saranno ovviamente presenti anche immagini dell'implementazione stessa.

Il documento è strutturato nel modo seguente: nel primo capitolo verrà descritta la piattaforma di e-assessment LibreEOL, nell'ambito della quale il lavoro svolto si colloca; nel secondo capitolo verranno descritte le tecnologie utilizzate; nel terzo capitolo viene descritta l'implementazione del progetto; infine nel quarto capitolo il processo di sviluppo per l'intero progetto.

Capitolo 1

LibreEOL e importanza dei report

Il progetto LibreEOL inizia nel 2000, presso il laboratorio di High Performance Computing del dipartimento di Matematica e Informatica dell'Università di Perugia dal Professor Osvaldo Gervasi e i suoi collaboratori. LibreEOL è un software open source realizzato per creare e gestire esami e certificazioni online. Il software viene sviluppato come **applicazione web** ed è in grado di generare diversi tipi di test, all'interno dei quali è possibile inserire **dieci** diversi tipi di domande dove il docente può richiedere diversi tipi di risposta agli utenti, in base alle necessità richieste dalla materia o dal tipo di test.

Le domande in LibreEOL possono essere divise per **argomento**, per **tipologia** e per **difficoltà**. La difficoltà è direttamente assegnabile dal docente, mediante la sua interfaccia utente con i suoi privilegi dedicati. Le **difficoltà** possono essere di tre livelli e sono rispettivamente: facile, medio e difficile. LibreEOL ha anche un supporto **multilingua** sia per quanto

riguarda l'intera interfaccia del sito, sia per le domande poste nell'esame, queste infatti possono avere il proprio testo definito dal docente e poi tradotto nelle lingue necessarie per permettere a tutti gli studenti di svolgere l'esame senza alcun problema di barriera linguistica. Nei capitoli successivi verrà anche descritto più in particolare il funzionamento del sistema di traduzione implementato in LibreEOL.

Il software permette non solamente lo svolgimento di **esami online**, ma anche quello di corsi di formazione. Si possono svolgere gli esami sia da **computer** che da **tablet**, utilizzando l'ambiente Amazon ECS, rinomato per performance e affidabilità.

L'utilizzo di LibreEOL porta vantaggi quanto a **neutralità della correzione**, vista l'assenza di parametri soggettivi in gioco al momento della valutazione; i test sono bilanciati tutti ugualmente, estraendo domande da un set definito dal docente in precedenza. Il software permette inoltre una grande semplificazione nella gestione di molti studenti e, come già dimostrato, rende possibile lo svolgimento di esami in via **telematica** e non solamente in presenza. LibreEOL, come accennato in precedenza, assegna un numero di domande scelto dal docente estraendole casualmente da un set predeterminato di domande. Ogni test sarà quindi bilanciato dal software avendo lo stesso quantitativo di domande per **difficoltà**. Al momento della consegna della prova, il software corregge **autonomamente** l'esame e lascia al docente la possibilità di controllare la correzione e di vedere le risposte date dallo studente.

Per permettere l'uso del sistema in remoto, si è fatta necessaria l'implementazione di tecniche di controllo per evitare abusi del sistema e richiedere

aiuto da terzi durante lo svolgimento dell'esame. Il software è infatti in grado di intercettare tentativi di cambio pagina per consultare fonti online e richieste di aiuto a persone presenti nella stessa stanza dell'esaminato. LibreEOL è rilasciato con licenza **GPL 3** e la versione su cui è stato svolto il progetto, non quella attualmente online e in uso, è disponibile e scaricabile su **GitHub**.

1.1 Importanza dei report

Il progetto è basato sull'implementazione di **report** in seguito allo svolgimento di un esame. I report implementati permettono la visualizzazione nel dettaglio dello svolgimento di un esame di uno specifico studente rispetto alla media e al totale degli esaminati. I report hanno un'importanza fondamentale per capire le lacune comuni degli studenti, le quali potrebbero essere dovute alle cause più diverse. Il sistema di reportistica ha quindi lo scopo di individuare questo tipo di lacune che potrebbero essere dovute ai metodi di insegnamento o alla difficoltà di apprendimento per alcuni argomenti trattati nei corsi presi in esame.

Il **reporting** non è presente solamente nella sfera riguardante l'educazione e la formazione, è infatti ben più utilizzato nelle aziende per comunicare informazioni come ad esempio l'andamento generale dell'azienda, l'andamento di specifici soggetti, l'assunzione di nuovi impiegati, la comunicazione con la dirigenza e molte altre ancora. Il **reporting** è quindi più nello specifico l'insieme dei documenti destinati ad andare a fornire informazioni sull'andamento della gestione, focalizzandosi sui possibili interventi applicabili o necessari per una migliore riuscita dell'andamento globale dell'azienda

o di singoli soggetti. L'obiettivo primario dei report è quello di raggiungere uno o più **KPI** (key performance indicator), una misura quantificabile nel tempo di una certa prestazione, fondamentali per comprendere l'andamento di una certa attività [1]) e gli obiettivi prefissati dall'azienda.

Fondamentale per un sistema di reportistica è la raccolta dei **dati** e come questa viene svolta. Nel caso specifico del progetto i dati erano già disponibili nel database, in quanto i report operano su test svolti dagli studenti, ma nel caso di un'azienda o di una qualsiasi altra misurazione la raccolta dei dati e la sua tempestività sono fondamentali per la modifica e implementazione di miglioramenti all'interno dell'ambito in cui i report vengono svolti.

I report sono anche suddivisibili in base a chi li riceve, possiamo infatti dividerli ad esempio come **report direzionali** se sono indirizzati alla direzione di un'azienda o in questo caso a dei docenti nei confronti degli utenti o altrimenti come **report operativi** se sono destinati ai **responsabili**, per esempio per mostrare loro uno scostamento rispetto alla media o agli obiettivi richiesti.

Esistono più tipi di report, gli **assessment report** sono uno di questi. Gli **assessment report** sono delle valutazioni utilizzate per stimare le capacità di un individuo o di una o più persone. Questo tipo di report vengono utilizzati praticamente in ogni industria e possono essere basati su diversi tipi di dati. Questo tipo di report può fornire testi o immagini visive per permettere la comprensione dei risultati ottenuti dall'esaminato basandosi esclusivamente sulle risposte fornite. Un esempio di report può essere un **personality assessment**, che consiste in un insieme di domande

basate sulla propria personalità. Questo tipo di report può fornire come risposta un **grafico radar** che mostra il tipo di personalità dell'esaminato, rendendo così più semplice trovare un impiego per un eventuale esaminatore incaricato di assegnare l'esaminato ad un certo reparto. Un assessment report può essere formato da più componenti come ad esempio:

- **Obbiettivo**, indica le finalità del report e aiuta chi lo legge a comprendere come valutarne i risultati.
- **Metodologia**, indica i metodi usati per valutare gli esaminati e determinarne i risultati. Indicando la metodologia si comprende al meglio come vengono assegnati i risultati finali.
- **Struttura dei risultati**, i risultati vengono riportati in modo che siano il più comprensibili e adatti possibili, ad esempio possono essere mostrati come testo, risultato numerico o grafici.

Nell'implementazione specifica si è scelto l'utilizzo di **grafici interattivi** che rendono molto bene l'idea dei risultati ottenuti dagli studenti. Un'altra alternativa sarebbe potuta essere la redazione di **report testuali** che indicassero con poche righe l'andamento di uno studente in base ai risultati ottenuti, implementabile anche mediante qualche tipo di intelligenza artificiale.

Capitolo 2

Tecnologie Utilizzate

Più **tecnologie** sono state utilizzate nel corso del progetto, queste vengono divise in base all'utilizzo che ne è stato fatto ed al relativo contesto operativo. Le due macro categorie suddividono le tecnologie in base al compito che queste devono svolgere e sono rispettivamente: **Front-end** e **Back-end**. Per lo sviluppo Front-end sono stati utilizzati i tre linguaggi più standard per lo scopo: **HTML**, **CSS** e **JavaScript**. Per lo sviluppo **Back-end** sono stati utilizzati i linguaggi **PHP** e **MySQL**. Il primo per computazioni lato server mentre il secondo per la gestione del database del sistema. Da menzionare anche l'utilizzo di due librerie **JavaScript** fondamentali per lo sviluppo del progetto:

- "**Chart.js**", usata per lo sviluppo di tutti i grafici presenti nella sezione di report implementata nel il progetto.
- "**jQuery**", libreria JavaScript utilizzata per semplificare la scrittura di codice in, appunto, JavaScript.

Per reperire le informazioni dal database presente nel server al frontend sono state utilizzate chiamate **AJAX** e notazione **JSON**. Di seguito verranno descritte più nello specifico tutte le tecnologie usate e in quale parte del progetto queste vengono impiegate e a quale scopo.

2.1 HTML

HTML, acronimo di "HyperText Markup Language" è il **markup language** standard per pagine web¹. Un **markup language** è un sistema di encoding formato da un insieme di simboli utilizzato per formattare un documento di testo e controllarne la sua struttura e relazione fra le sue parti. Un **markup language** è quindi un insieme formale di **regole** che facilita sia umano sia macchina nella comprensione di come il testo dovrebbe essere mostrato a schermo. **HTML** viene utilizzato in coppia con il linguaggio **CSS**, utilizzato per modificare lo **styling** delle pagine. Anche in questo progetto è stato utilizzato **CSS** che verrà trattato più nello specifico in seguito.

Le informazioni in **HTML** vengono trasmesse mediante dei marcatori detti **tag**, che si contraddistinguono poiché sono inclusi tra parentesi angolari. Essendo **HTML** un linguaggio di **markup** e non un linguaggio di programmazione, non possiede i cosiddetti meccanismi condizionali che permettono azioni basati su condizioni vere o false. In HTML non sono presenti neanche i classici costrutti iterativi che permettono la reiterazione di azioni sulla base di condizioni. A causa di queste mancanze parliamo quindi di un linguaggio **dichiarativo**, poiché indica come vadano mostrate le informa-

¹<https://en.wikipedia.org/wiki/HTML>

zioni a schermo, con quali qualità e in che sequenza. Il linguaggio HTML viene utilizzato nel progetto per mostrare i contenuti a schermo e usato sia nelle porzioni di codice **back-end** sia in quelle **front-end**, lavorando quindi insieme ai linguaggi **PHP** e **JavaScript**.

2.2 CSS

CSS, acronimo di Cascading Style Sheets¹ è uno style sheet language, usato per personalizzare la resa grafica di documenti scritti in un **markup language** come ad esempio **HTML** o **XML**. In **CSS** è possibile specificare caratteristiche grafiche di elementi all'interno del documento. Ad esempio è possibile modificare font, dimensione, colore del testo. Mediante l'uso di **CSS** è possibile applicare lo stesso stile a più pagine contemporaneamente, così da risparmiare ripetizioni nel codice e rendendo possibile la modifica di molti componenti senza dover modificare più documenti diversi fra loro. Nell'acronimo **CSS**, cascading sta a significare che i fogli di stile vengono applicati a **cascata**, questo significa che un elemento è soggetto a diverse regole che sono tutte valide, ma prevale solamente la più specifica. In CSS definiamo la **specificità**, quell'algoritmo utilizzato dai browser per determinare quale dichiarazione CSS è la più rilevante per un elemento e quindi quale applicargli. L'algoritmo calcola il **peso** di un selettore [7] CSS per determinare quale regola, fra alcune concorrenti, deve essere applicata all'**elemento**². In ordine, dal più al meno specifico il peso assegnato ai selettori CSS:

¹<https://en.wikipedia.org/wiki/CSS>

²<https://developer.mozilla.org/en-US/docs/Web/CSS/Specificity>

- **ID**, selettori mediante id, come ad esempio "#example".
- **CLASS**, selettori mediante classe, come ad esempio ".exampleClass".
- **TYPE**, selettori per tipo, come ad esempio "p", "h1".
- **No value**, per selettori come il selettore universale (*) o pseudo-classi come :where().

Piccola digressione necessaria è la definizione di **DOM** [4]. Acronimo di **Document Object Model**, è una API che rappresenta la struttura della pagina così da rendere possibile l'accesso ai suoi elementi a programmi. Si può, mediante il DOM, modificare la struttura, lo stile e il contenuto dei vari oggetti presenti all'interno della pagina. Il DOM rappresenta inoltre la pagina come **oggetti** e **nodi** così da rendere intuitivo l'accesso alle componenti ai vari programmi.

I **selettori css** permettono quindi di applicare stili in modo condizionale, basando la scelta sul tipo di elemento presente nel **DOM**. Come già accennato in precedenza è possibile raggruppare elementi del **DOM** mediante classi personalizzate per applicare stili a più elementi senza ripetizioni o altrimenti mediante l'uso di ID. Gli **ID** sono identificatori univoci applicati ad elementi presenti nel **DOM**, utilizzati per distinguere in modo univoco elementi presenti nella pagina. Gli **ID** sono anche selettori più specifici rispetto a selettori di classe in **CSS**.

2.3 JavaScript Chart.js e jQuery

JavaScript è un linguaggio di programmazione **multi paradigma** e orientato agli eventi, utilizzato nella maggior parte dei siti web. JavaScript può

essere utilizzato sia lato **client** che lato **server** (mediante l'uso di Node.js è possibile l'utilizzo lato server). Nel progetto **JavaScript** è stato utilizzato lato client front-end, per la manipolazione delle pagine e l'interazione dell'utente con il sistema. Un altro utilizzo fondamentale di JavaScript all'interno progetto è l'interazione con il **server** per reperire informazioni necessarie richieste dall'utente da mostrare a schermo. Per svolgere questo compito verranno utilizzate più chiamate **AJAX** nel corso del codice realizzato per lo sviluppo del progetto.

Per la creazione dei grafici del progetto si è fatto utilizzo della libreria "**Chart.js**", una libreria in grado di creare **grafici dinamici**, esteticamente accattivanti e utilizzati per mostrare informazioni utili riguardo le prestazioni dei vari studenti presenti nel sistema. Mediante **Chart.js** è possibile personalizzare i grafici come meglio si addice al loro utilizzo e la libreria offre una moltitudine di grafici perfetti per la rappresentazione di dati relativi a punteggi e prestazioni. La libreria si assicura anche che i grafici siano **responsive**, cioè in grado di adattarsi a più tipi di schermi come quelli di smartphone e tablet ormai molto popolari per la navigazione web. I grafici vengono rappresentati all'interno di oggetti **canvas HTML** e la libreria è compatibile con la maggior parte dei **framework** di successo come React, Vue, Svelte e Angular ¹.

L'altra libreria utilizzata nel progetto è "**jQuery**", progettata per rendere più semplice la navigazione di un documento, selezionare elementi del **DOM**, creare animazioni, gestire eventi e sviluppare applicazioni AJAX. Nello specifico le funzionalità più utilizzate all'interno del codice sono state:

¹<https://www.chartjs.org/docs/latest/>

- **Chaining**, solitamente un metodo jQuery ritorna un oggetto jQuery, in questo modo è possibile chiamare più metodi in sequenza, potendo fare quindi molte chiamate in una sola riga rendendo il codice di più facile comprensione e scrittura.
- **AJAX**, la sintassi jQuery risulta molto più leggibile rispetto a una chiamata scritta con codice sviluppato solamente in JavaScript, le richieste sono anche compatibili su più browser.
- **Selezione Elementi del DOM**, il concetto più semplice e di base usato in jQuery e di conseguenza anche uno dei più utilizzati, rende la selezione di elementi all'interno del codice HTML molto semplice e veloce. Per selezionare viene usato l'operatore **dollaro** all'interno del quale sarà possibile selezionare mediante svariate modalità, le più utilizzate sono state mediante **classe**, **ID** e **attributo**.

2.4 PHP

PHP, acronimo di **Hypertext Preprocessor**, è un linguaggio di scripting interpretato, concepito per la programmazione di pagine web dinamiche. Attualmente viene utilizzato per sviluppare applicazioni web lato server, come nel caso di questo progetto. Un altro impiego di **PHP** può essere la scrittura di script a riga di comando o per applicazioni **stand alone** con interfaccia grafica. Grazie anche alla diffusione del **CMS** (content management system) Wordpress, scritto interamente in **PHP**, la popolarità del linguaggio è salita di molto negli ultimi anni. PHP ha diverse versioni disponibili per l'utilizzo che spaziano dalla meno recente versione 3 fino

all'ultima rilasciata versione 8, nello specifico del progetto è stata utilizzata la versione 7.4. All'interno del progetto **PHP** viene utilizzato, come già detto, lato **server** per richiamare query al database e per manipolare i dati che verranno poi passati al codice **front-end** per la visualizzazione a schermo destinata all'utente. LibreEOL segue un modello tipo **MVC** [2], model-view-controller il quale è un pattern architetturale che implementa appunto:

- **Model**, il quale contiene i metodi di accesso ai dati.
- **View**, la quale visualizza i dati all'utente e gestisce l'interazione fra l'utente e l'infrastruttura sottostante. È possibile avere più viste dello stesso dato.
- **Controller**, il quale riceve i comandi dell'utente attraverso le view e reagisce eseguendo delle operazioni che possono interessare il model che generalmente portano ad un cambiamento di stato della view.

Seguire un pattern architetturale permette di strutturare il codice secondo una logica ben precisa che rende più semplice la manutenzione e la comprensione di quest'ultimo. In particolare il pattern MVC permette la divisione di **ruoli** nel codice che favorisce il riutilizzo e aiuta nel trovare soluzioni per problemi comuni, incrementando scalabilità, uniformità e rendendo la manutenzione del codice più semplice. La suddivisione applicata nel modello **MVC** permette anche la semplificazione nella divisione del lavoro; nel caso in cui il software sia affidato ad un team composto da più persone, sarà infatti facile assegnare lavoro basato su competenze specifiche. Il modello MVC porta anche il vantaggio che nel caso avvenga un cambio di tipo di

database, sarà possibile cambiare solamente il modello piuttosto che tutto il codice.

2.5 MySQL

MySQL è un **RDBMS** (relational database management system) **open source**, il cui nome deriva dall'unione di "My", nome della figlia del co-founder Michael Widenius, e SQL, acronimo di structured query language.

Un database relazionale struttura i suoi dati in tabelle, dette **relazioni**, che possono interagire fra loro. Mediante l'uso di SQL il programmatore è in grado di interagire con queste relazioni e può creare, interrogare o modificare il database presente. In aggiunta un RDBMS come MySQL fa uso di un **sistema operativo** per mantenere nella memoria della macchina i dati presenti nel database relazionale.

2.6 AJAX

Acronimo di **Asynchronous JavaScript And XML** ¹ è una tecnica di sviluppo software in cui si fa uso dell'oggetto "XMLHttpRequest" per comunicare con il server. Le informazioni possono essere inviate/ricevute in diversi formati tra cui: **JSON**, **XML**, **HTML** e file di testo, nel caso specifico del progetto si fa uso del formato di tipo **JSON**. La caratteristica più importante della tecnologia **AJAX**, e che ne fa prediligere l'uso, è il funzionamento asincrono che permette la comunicazione e l'aggiornamento dei dati senza la necessità di aggiornare la pagina. Le caratteristiche fonda-

¹https://developer.mozilla.org/en-US/docs/Web/Guide/AJAX/Getting_Started

mentali della tecnologia AJAX sono quindi la possibilità di fare richieste al server senza dover aggiornare la pagina e la possibilità di lavorare con dati ricevuti dal server.

2.7 JSON

Acronimo di **JavaScript Object Notation**¹, è un formato testuale usato per lo scambio di dati. Il formato **JSON** viene spesso utilizzato in JavaScript e molte applicazioni web, inoltre è un formato supportato da molti linguaggi di programmazione.

JSON può rappresentare numeri, stringhe, booleani, valori null, array e oggetti composti da valori di questo tipo, ma non può nativamente rappresentare strutture dati più complesse come ad esempio funzioni, espressioni regolari o date. Nello specifico del progetto, il formato JSON è stato utilizzato per lo scambio di dati mediante AJAX fra client e server.

2.8 XML

Acronimo di **Extensible markup language** è, come HTML, un linguaggio di markup utilizzato per contenere informazioni. Utilizza un sistema di regole per la scrittura di dati leggibile sia per umani che per persone. Questo formato viene spesso utilizzato per rappresentare strutture dati in web service, per trasmettere informazioni via API o per rappresentare informazioni specifiche come nel caso del progetto. XML è stato infatti utilizzato nella parte della **traduzione** del sito, **LibreEOL** prevede l'utilizzo di un

¹<https://developer.mozilla.org/en-US/docs/Glossary/JSON>

file di questo tipo per ogni lingua che necessita di traduzione ed assegna ad ogni costrutto da tradurre un ID, che viene poi convertito in variabili **Java-Script** e **PHP** dopo una procedura di compilazione avviata da un account di ruolo Admin.

Capitolo 3

Implementazione dei report

In questo capitolo viene descritta nel dettaglio l'implementazione del progetto. Verranno anche descritte eventuali difficoltà incontrate e come queste sono state affrontate.

3.1 CoachingReport View

Come detto in precedenza il sistema utilizza il pattern **MVC**, quindi in questa sezione verrà descritta con precisione la view principale utilizzata per l'implementazione del progetto.

La **view**[3] rappresenta il componente che interagirà con l'utente e fornirà a questo i dati da visualizzare a schermo e ne riceverà gli input. La view utilizzata per il progetto è stata creata su misura ed è "**coachingreport.php**", alla quale si accede dopo un click su un bottone riportante "**coaching report**" nella tabella dei risultati di un esame. Come prima cosa questa view visualizza a schermo header, footer e action bar standard di LibreEOL. Per visualizzare a schermo l'action bar viene chiamata la funzione **printMenu**

Implementazione dei report

Visualizzati 15 studenti registrati

SEARCH: [Assicurati](#)

	Nome	E-mail	Ora di inizio	Ora di fine	Tempo usato	Punteggio	Punteggio finale	Gestisci
	anastasi diego	diego@gmail.com	2023-01-06 18:38:01	2023-01-06 18:39:45	00:01:44	7/10	8/10	
	cugliesi giacomo	giacomo@gmail.com	2023-01-06 18:33:14	2023-01-06 18:34:31	00:01:17	7/10	7/10	
	caligaris federica	federica@gmail.com	2023-01-06 18:17:32	2023-01-06 18:18:55	00:01:23	4/10	4/10	
	castellani federico	caste@gmail.com	2023-01-06 18:28:03	2023-01-06 18:29:33	00:01:30	7/10	8/10	
	cavallaghi paolo	paolo@gmail.com	2023-01-06 18:30:45	2023-01-06 18:32:06	00:01:21	6/10	7/10	
	fafa lorenzo	lorenzo@gmail.com	2023-01-06 18:15:01	2023-01-06 18:16:31	00:01:30	4/10	5/10	
	galli agnese	agnese@gmail.com	2023-01-06 18:35:41	2023-01-06 18:37:06	00:01:25	6/10	6/10	
	gambacorte michele	michele@gmail.com	2023-01-06 18:32:03	2023-01-06 18:34:50	00:02:47	0/10	0/10	
	menecarrelli andrea	meneco@gmail.com	2023-01-06 18:16:19	2023-01-06 18:17:56	00:01:37	4/10	4/10	
	pagliuca gela	gela@gmail.com	2023-01-06 18:19:56	2023-01-06 18:21:52	00:01:56	4/10	5/10	
	potravni francesco	potravni@gmail.com	2023-01-06 18:25:23	2023-01-06 18:26:57	00:01:34	4/10	5/10	
	rossi federico	federico@gmail.com	2023-01-06 18:23:55	2023-01-06 18:25:35	00:01:40	5/10	5/10	
	rossi mario	mario@gmail.com	2023-01-06 18:22:41	2023-01-06 18:23:47	00:01:06	6/10	6/10	
	sensi alessandra	alessandra@gmail.com	2023-01-06 18:37:56	2023-01-06 18:39:00	00:01:04	7/10	7/10	
	stefano stefano	stefano@gmail.com	2023-01-06 18:39:38	2023-01-06 18:40:33	00:00:55	5/10	6/10	

PREVIOUS 1 NEXT

[Chiudi](#) [Coaching Report](#) [Report](#) [Aggiorna](#)

Figura 3.1: Vista degli studenti che hanno sostenuto l'esame. Per accedere ai report è presente un bottone in basso a destra.

definita in "essential.php". Questa funzione mostra a schermo un menù basato sul ruolo del client, questo poiché in LibreEOL gli utenti sono divisi in diversi ruoli, ognuno rappresentato da una lettera nel sistema:

- **Teacher**, rappresentato dalla lettera "t".
- **Examiner** rappresentato dalla lettera "e".
- **eTeacher** rappresentato dalla lettera "er".
- **admin** rappresentato dalla lettera "a".
- **student** rappresentato dalla lettera "s".
- **adminTeacher** rappresentato dalla lettera "at".
- **guest** rappresentato dal simbolo "?".

Ogni ruolo ha diversi privilegi, la funzione quindi, riconoscendo il ruolo dalla lettera assegnata all'utente, è in grado di visualizzare un menù che ha le funzionalità previste per quel tipo di utente. Header e footer vengono visualizzati utilizzando le rispettive chiamate di funzione "**renderHeader()**" e "**renderFooter()**" già ovviamente presenti nel software.

In questa view vengono creati due **dropdown**, utilizzati dall'utente per scegliere la visualizzazione di report per l'insieme degli studenti o per il singolo studente. Entrambi vengono creati utilizzando il tag html dedicato `<select>`, al quale vengono forniti i dati (in questo caso i nomi degli studenti che hanno svolto l'esame) dopo un'interrogazione al database mediante l'uso di **PHP**.

La pagina contiene anche l'id dell'esame utilizzato nell'esecuzione della query "qExamRegistrationsList" che, prendendo come parametro l'id in questione, restituisce la lista dei partecipanti all'esame. Il secondo **dropdown** della pagina ha solamente due opzioni che permettono la scelta tra la visualizzazione delle risposte e relativi dati a riguardo o la visualizzazione dei grafici dello studente. Il primo dropdown contiene anche l'opzione "tutti", la quale permette la visualizzazione di grafici riguardanti le performance globali di tutti gli studenti.

Nella view viene anche creato il bottone "**submit**", il quale al momento del click renderà visibili e nasconderà diversi elementi della pagina in base alle scelte nei dropdown dell'utente. Infine in questa view vengono anche istanziate le **canvas** dedicate alla visualizzazione dei grafici in chart.js, la libreria adibita alla creazione dei grafici dinamici riportanti le informazioni riguardo l'esame preso in considerazione.



Figura 3.2: I due dropdown presenti nella pagina.

3.2 Controller e Azioni

In LibreEOL l'URL della pagina ha una struttura ben definita che segue il pattern **MVC**, infatti un esempio di URL presente nel software è `http://localhost/public/index.php?page=exam/coachingreport`. Tutti gli URL iniziano per **index.php** e sono seguiti da **page=** che dopo indica il **controller** seguito a sua volta da **/coachingreport** che è la view.

Il controller che viene maggiormente utilizzato nel progetto è l'**Exam-Controller**. In questo controller vengono registrate tutte le azioni più importanti per l'implementazione dei report.

Nel sistema LibreEOL, sempre seguendo il pattern MVC, vengono inserite delle azioni all'interno di ogni controller. Il controller ha l'incarico di controllare che l'utente abbia i diritti necessari per svolgere determinate azioni, per questa operazione vengono utilizzate le **access rules** e delle funzioni che vengono ereditate dalla classe Controller, che tutti i controller estendono. Una funzione ereditata è **getAccess**, la quale controlla se l'utente ha accesso a una determinata azione o meno, ritornando true se ne ha accesso o false altrimenti. Per registrare un'azione all'interno del controller si fa uso di una funzione chiamata **accessRules**, all'interno della quale si specificano in **array** i nomi delle azioni e i ruoli che possono eseguirle, in questo modo non si rischiano vulnerabilità riguardanti permessi agli utenti.

Nella funzione **accessRules** è possibile anche negare il diritto di eseguire alcune azioni a un determinato ruolo, utilizzando la parola 'deny', opposto di 'allow' che permette invece l'utilizzo di azioni ai ruoli.

```
1  private function accessRules(){
2      return array(
3          array(
4              'allow',
5              'actions' => array('exampleAction1'),
6              'roles'   => array('a'),
7          ),
8          array(
9              'allow',
10             'actions' => array('exampleAction2','
exampleAction3'),
11             'roles'   => array('t','e'),
12         ),
13         array(
14             'allow',
15             'actions' => array('exampleAction4'),
16             'roles'   => array('s'),
17         ),
18         array(
19             'deny',
20             'actions' => array('*'),
21             'roles'   => array('*'),
22         ),
23     );
24 }
```

Codice 3.1: Funzione accessRules

Nel controller "examController" sono inserite tutte le più importanti operazioni **backend** che vengono svolte dall'implementazione, infatti in questa classe possiamo trovare l'esecuzione delle query e la manipolazione dei dati dopo che questi sono reperiti.

3.3 Funzioni Exam Controller

Tutte le funzioni all'interno dell'Exam Controller sono funzioni **private** che, come detto in precedenza, sono registrate nelle access rules. Le azioni implementabili non saranno utilizzabili dagli studenti, ma essendo solamente report per docenti, saranno accessibili solamente dal ruolo di **docente** o **superiori**. Tutte le funzioni implementate sono private e forniscono un endpoint per eseguire delle chiamate AJAX per le interrogazioni al database. Infatti nella maggior parte delle funzioni viene creata una variabile contenente il riferimento a una istanza della classe sqlDB, usata per eseguire le query all'interno del database MySQL. Ora verranno mostrate alcune delle azioni implementate all'interno di questo **controller**.

3.4 Funzioni nello specifico

3.4.1 Risposte di uno studente

```
1     private function actionStudentanswers(){
2         $db = new sqlDB();
3         $db->qGetanswers($_POST['name'],$_POST['surname'],
4         $_POST['idExam']);
5         $result = array();
6         while($answer = $db->nextRowAssoc()){
7             array_push($result, $answer);
8         }
9         echo json_encode($result);
10    }
```

Codice 3.2: Funzione actionStudentanswers

Questa funzione reperisce dal database le risposte di un dato studente, partendo dal suo **nome** e **cognome** a un dato esame, usando l'id specifico dell'esame. Tutti i parametri, come è facile vedere, vengono passati utilizzando il metodo **POST** del protocollo **HTTP** dalla parte front-end. Il nome e il cognome dello studente sono scelti dall'utente mediante il **drop down** descritto nelle sezioni precedenti, mentre l'id dell'esame viene reperito nel momento in cui il docente sceglie di quale esame ottenere i report. La funzione ritorna un array che viene poi tradotto usando la codifica **JSON**, mediante lo specifico metodo PHP "json_encode", il quale, passando un valore, ritorna una stringa in codifica JSON, necessaria per la comunicazione front-end con JavaScript. Nella funzione è presente un ciclo **while**, poiché le informazioni ritornate dalla query vengono estratte utilizzando il metodo nextRowAssoc(). Il metodo pubblico nextRowAssoc() è proprio della classe

sqlDB.php e ritorna la prossima riga di un array associativo, in questo caso l'array è quello del risultato della query. Nella funzione specifica implementata quindi si estraggono righe dall'array risultato fino a che non sono state tutte estratte.

3.4.2 Funzione per ottenere la media da un insieme di domande

```
1     private function actionAveragefromset(){
2         $db = new sqlDB();
3         $names = $_POST['array'];
4         $result = 0;
5
6         for($i=0; $i<count($names); $i++){
7
8             $db->qGetanswers($names[$i][0], $names[$i][1],
9             $_POST['idExam']);
10
11             while($answer = $db->nextRowAssoc()){
12                 $result += $answer['score'];
13             }
14         }
15
16         echo json_encode(($result/count($names)));
17     }
```

Codice 3.3: Funzione actionAveragefromset

In questa e le seguenti funzioni non verranno trattati nuovamente concetti già espressi in funzioni o sezioni precedenti per evitare ridondanza. Questa

funzione mostra l'utilizzo fatto di PHP, non solo usato per reperire i dati, ma anche per elaborarli e fornirli già lavorati al front-end. In questa funzione i dati vengono reperiti come in precedenza, ma viene fatta un'operazione di somma di tutti i risultati ottenuti dagli studenti e una divisione per il loro numero, ottenendo così la media che verrà direttamente passata al frontend senza necessità di calcolarla lato client. Il parametro "names" passato in questa funzione è la lista di tutti i nomi degli studenti iscritti a questo esame, che coincide con i dati presenti nel **drop down** degli studenti presente nella pagina mostrata al client.

3.4.3 Funzione per ottenere il numero di domande giuste per categoria

```
1     private function actionGetanswerscorecategory(){
2         $db = new sqlDB();
3
4         $db->qGetQuestionCountCat($_POST['name'], $_POST['
5             surname'], $_POST['idExam']);
6
7         $correctAnswersCount = array();
8
9         while($answer = $db->nextRowAssoc()){
10             array_push($correctAnswersCount, $answer);
11         }
12
13         echo json_encode(["correctAnswers" =>
14             $correctAnswersCount]);
```

```
}
```

Codice 3.4: Funzione `actionAverageFromset`

Questa funzione restituisce al **frontend** un oggetto **JSON** all'interno del quale troviamo in numero di domande giuste per categoria ottenute da uno studente. Le categorie sono: vero/falso, risposta aperta, risposta numerica, scelta multipla, scelta singola e si/no. In questa funzione viene ritornato un array associativo all'interno del quale viene aggiunta una chiave chiamata "correctAnswers". Un array associativo è un array che, invece di utilizzare indici numerici per accedere ai suoi elementi, usa chiavi che permettono l'accesso ai dati. In questo caso per accedere in modo più leggibile e di facile comprensione, a livello di codice, si è preferito l'aggiunta di una key "correctAnswers", abbastanza auto esplicativa al momento della lettura.

3.5 Query e classe `sqlDB.php`

Le query sono implementate nella classe `sqlDB.php`, la quale è adibita all'interrogazione diretta del database MySQL. Le query sono state progettate per essere protette da attacchi di tipo **sqlInjection**. L'attacco **sqlInjection** è una vulnerabilità che permette all'attaccante di eseguire codice SQL non previsto all'interno dell'applicazione. L'attaccante potrebbe ad esempio eseguire una query per eliminare database o relazioni, creando grossi problemi all'interno del sistema sotto attacco [6]. L'attacco è possibile nel momento in cui l'utente può inserire del testo che viene direttamente inviato al database, se il codice non è sicuro e l'attaccante inserisce sintassi **SQL** che verrà interpretata direttamente dal database divenendo così vulnerabile.

Questa vulnerabilità non dipende dalle tecnologie utilizzate, ma solamente dalla sintassi utilizzata e dall'accortezza dello sviluppatore nei confronti di questo tipo di attacco. Più avanti verranno trattate nello specifico le query e il loro codice con le loro versioni sicure e non da attacchi SQL Injection.

Ogni query viene inserita quindi all'interno della classe **sqlDB** come una funzione ed ognuna di esse ha come prefisso la lettera "q". Ognuna di queste ritorna un valore **vero** o **falso**, il quale dipende dal risultato ottenuto dalla query, infatti se la query ritorna almeno un numero maggiore di zero righe come risultato ritorna true, altrimenti in caso di exception o zero righe ritorna false. In ogni query per prima cosa viene invocata la funzione **connect**. Questa funzione permette la connessione al database facendo uso di un oggetto mysqli (estensione utilizzata per comunicare ad un database SQL, porta dei vantaggi rispetto alle vecchie versioni in quanto a sicurezza e prestazioni) e ritorna l'oggetto mysqli che viene poi assegnato come prima operazione all'interno della funzione della query. I parametri di connessione utilizzati all'interno di questa funzione (salvati nel file config.php) sono stati modificati nelle fasi iniziali di setup del progetto per permettere la connessione in locale a un server web il **XAMPP**. Tutto il progetto è stato testato e sviluppato in locale utilizzando questa tecnologia.

Un'altra porzione fondamentale di codice, presente in ogni query, è il blocco **try catch** che esegue la query vera e propria, nel caso in cui la query vada a buon fine si ritorna il valore true, altrimenti se si finisce nel blocco catch, perché una eccezione è stata sollevata, allora si ritorna false e si scrive in un file di **log** l'eccezione sollevata.

All'interno del log si scriverà, non solo l'errore sollevato, ma anche la

costante predefinita `__FUNCTION__`. PHP infatti fornisce una moltitudine di costanti predefinite, in questo caso il valore corrisponde al nome della funzione che ha sollevato l'eccezione. La struttura delle query è standardizzata, le differenze sostanziali si trovano nel contenuto dell'interrogazione e di come viene scritto il codice **SQL** per ottenere il risultato desiderato.

Nelle prime fasi di sviluppo la struttura del database di **LibreEOL** è stata studiata mediante un grafico EER, generato utilizzando la funzione di Reverse-engineering del software SQL Workbench. Eseguire interrogazioni senza conoscere a pieno la struttura del database sarebbe risultato altrimenti impossibile. **SQL workbench** è un software progettato per visualizzare database sviluppati in **MySQL**, l'uso del software nel progetto è stato esattamente questo. Non conoscendo la struttura del database inizialmente, usando **XAMPP**, è stata analizzata la struttura mediante la funzione per visualizzare il grafico del database di phpMyAdmin. La visualizzazione non era sufficientemente chiara e quindi si è optato per la rappresentazione **EER** del database con **MySQL Workbench** che ha dato il risultato in figura. **EER**, acronimo **enhanced entity relationship**, è un tipo di grafico utilizzato per rappresentare la struttura di un database relazionale. Con questa notazione vengono rappresentate tutte le tabelle, le loro chiavi e le relazioni che intercorrono fra loro.

La rappresentazione effettuata da **MySQL workbench** è dinamica e fa uso di animazioni per rendere più semplice la visualizzazione dello schema. Vengono infatti rappresentate in verde le chiavi primarie quando si passa il mouse su una determinata tabella e in giallo una eventuale chiave esterna presente nella tabella. **Primary key** in SQL identifica un vincolo per cui un

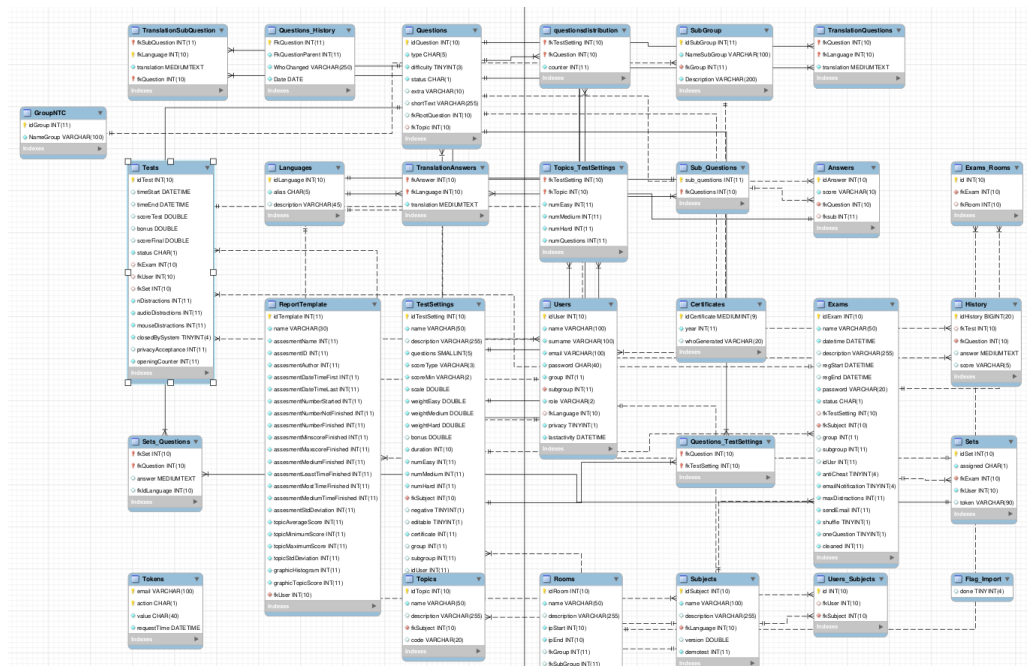


Figura 3.3: Grafico EER

attributo viene identificato come chiave primaria. Questo vincolo implica che la colonna non contenga mai valori ripetuti o nulli. Un esempio di chiave primaria all'interno della struttura LibreEOL è l'**id dello studente**, chiave utilizzata per identificare univocamente uno studente nel sistema. Nel db di LibreEOL le chiavi primarie sono rappresentate dagli id. Altro elemento fondamentale utilizzato nelle strutture dei database relazionali sono le **foreign key** o **chiavi esterne**. Una chiave esterna è una colonna che fa riferimento a una **chiave primaria** di un'altra tabella. Nella struttura del database di LibreEOL sono etichettate con la sigla **fk** che sta per **foreign key**.

Un altro concetto fondamentale utilizzato nella maggior parte delle query è l'operatore "**join**". L'operatore **join**, in SQL, è un operatore che permette

l'unione di righe da due tabelle diverse su una condizione. La condizione viene inserita dopo la parola chiave **ON**, questa tecnica è stata utilizzata spesso nel progetto, ad esempio se viene eseguita una query che restituisce gli id degli utenti, è possibile effettuare un join per ottenere informazioni riguardanti quegli utenti in ogni tabella che utilizza quegli id come chiave esterna.

Nello sviluppo del codice è stato fatto notare una **bad practice** corretta in corso d'opera che è l'utilizzo dell'operatore "*" nelle query SQL. Questo operatore, se posto dopo un "SELECT", permette di reperire tutte le colonne nella tabella selezionata che di per sé non è un male, ma in caso di refactoring o nel caso in cui un esterno dovesse leggere la query, farebbe più fatica a comprendere quali colonne vengono selezionate costringendolo a dover consultare la struttura del database. Per questa motivazione nell'ultimare il progetto sono stati rimossi tutti gli "*" dalle query nella file **Sqldb.php** e sostituiti con le colonne selezionate. Anche l'operatore **"AS"** è stato utilizzato più volte all'interno del progetto, questo operatore consente di rinominare tabelle e/o colonne all'interno della query. L'operatore **"WHERE"** è invece l'operatore condizionale utilizzato in SQL che insieme all'operatore **ON** usato nei join permette selezioni condizionali. Nel progetto sono state usate anche condizioni multiple che vengono valutate contemporaneamente mediante l'uso dell'operatore logico **AND**. L'operatore **COUNT** è un operatore che ritorna il numero di righe che soddisfano un certo criterio. Nel progetto questo operatore è stato usato più di una volta, ad esempio per contare il tipo di domande per un determinato test. Questo operatore può essere utilizzato su una singola colonna, co-

me nell'esempio appena descritto. Nel progetto è accompagnato anche da **"AS"** per rinominare la colonna risultante che altrimenti apparirebbe come `"count(column_name)"`. L'ultimo operatore degno di nota utilizzato nel progetto è **GROUP BY**, questo operatore permette di raggruppare righe che hanno lo stesso valore fra loro.

Di seguito si descriveranno le tabelle maggiormente utilizzate nel progetto. La tabella **Users**, già intuitivamente si comprende che è la tabella contenente le informazioni riguardanti gli utenti registrati nel sistema. Le informazioni reperite maggiormente nel corso del progetto sono nome, cognome, id. La tabella contiene anche informazioni quanto email del soggetto, password, un booleano rappresentante se l'utente ha accettato i termini di privacy e lingua utilizzata parte client.

Un'altra delle tabelle più importanti utilizzate nel progetto è la tabella **tests**. Questa tabella permette, insieme alla tabella **history**, la ricerca delle domande sottoposte a uno specifico studente e le sue risposte. La tabella contiene, come tutte, un id per identificare il test univocamente e altre colonne fondamentali come:

- **"Time Start e Time End"**, la differenza fra le due rappresenta la durata impiegata dall'esaminato per svolgere il test.
- **"FkUser e FkExam"**, due chiavi esterne che utilizzate in coppia formano una chiave candidata, cioè una coppia i cui singoli elementi non identificano un elemento univoco ma in coppia identificano un singolo test, in questo caso. Utilizzando queste due chiavi esterne si identificano nel progetto i risultati ottenuti dagli studenti.

- **scoreTest**, punteggio ottenuto da uno studente per quello specifico test.
- **nDistractions**, numero di distrazioni rilevate dalla webcam, parametro non utilizzato nel progetto.

Come detto in precedenza la tabella **test** è indispensabile in quanto viene utilizzato il suo id all'interno della tabella **history**. Questa tabella **history** infatti contiene una colonna la quale contiene come chiave esterna la chiave primaria id di test. La tabella **history** mantiene al suo interno id delle domande fatte in ogni test e la relativa risposta data dall'esaminato. Per individuare tutte le domande di un singolo esaminato entra in gioco la tabella **test** e il suo relativo id infatti, individuato l'id del test del relativo esame, basterà cercare tutte le righe che nella chiave esterna **fktest** hanno il valore del test ottenuto. Questo tipo di interrogazione fornirà la lista contenente gli id di tutte le domande e tutte le risposte fornite per ogni domanda date in un test. A questo punto per ottenere i dati effettivi delle domande e delle risposte occorre interrogare le rispettive tabelle. La tabella **questions**, utilizzando gli id delle domande, permette di ottenere:

- **shortText**, il testo della domanda proposto al candidato.
- **difficulty**, rappresentato da un intero che parte da un minimo di 1 (semplice) fino ad un massimo di 3 (difficile), campo usato spesso nel corso del progetto per la valutazione della difficoltà delle domande.
- **type**, tipo della domanda, anche questa colonna fondamentale per lo sviluppo del progetto, in quanto molti dei grafici si basano sul tipo e la quantità per tipo delle domande.

- **status**, che può variare tra tre stati, active, inactive ed error.

Ora viene descritta nello specifico qualche query. Le query sono state sviluppate inizialmente senza seguire le accortezze per renderle safe nei confronti di attacchi SQL Injection. Solo in seguito sono state corrette alla loro versione safe e definitiva.

```
1     public function qDurationTests($name, $surname, $idExam)
2     {
3         $this->mysqli = $this->connect();
4         $this->result = null;
5         $ack = true;
6
7         try {
8             $data = $this->prepareData([$name, $surname,
9             $idExam]);
10            $query = "SELECT
11            Users.name,
12            Users.surname,
13            (
14                Tests.timeEnd - Tests.timeStart
15            ) AS duration
16        FROM
17            Tests
18        JOIN Users ON Tests.fkUser = Users.idUser AND Users.name = '
19            $data[0]' AND Users.surname = '$data[1]' AND Tests.fkExam
20            = '$data[2]';";
21
22            $this->execQuery($query);
23            if ($this->numResultRows() > 0) {
24                $ack = true;
25            }
26        } catch (Exception $e) {
27            $ack = false;
28        }
29    }
```

```
22         }else{
23             $ack = false;
24         }
25     } catch (Exception $ex) {
26         $log->append(__FUNCTION__ . " : " . $this->
getError());
27         $ack = false;
28     }
29     return $ack;
30 }
```

Codice 3.5: Esempio codice Query

Questa funzione esegue una query per trovare il tempo impiegato da uno studente per svolgere l'esame su cui si basa il report. Non avendo nella tabella una colonna indicante direttamente il tempo impiegato, ma l'ora di inizio e l'ora di fine occorre eseguire una sottrazione tra ora di termine e ora di inizio. La query in questione ritorna, le colonne scelte sono inserite dopo il select, il nome, il cognome e la durata del test in secondi. Nel grafico corrispondente questa durata viene convertita in minuti, effettuando una semplice divisione per sessanta. Ora descriviamo più nello specifico questa query. Come in ogni query inizialmente i parametri erano passati senza avere alcun controllo, quindi utilizzando solamente i parametri dati al PHP. Ora, nella versione protetta delle query, i parametri sono passati alla funzione **prepareData**. Questa funzione permette l'escape di comandi SQL in modo che questi non vengano interpretati dal database come da eseguire. All'interno di **prepareData** viene passato un array contenente tutti i parametri ottenuti dal metodo post http, in questo caso tre, e restituisce un array della stessa lunghezza, che viene inserito all'interno della variabile

data. **ExecQuery** è la funzione adibita all'esecuzione della vera e propria query e viene passata la variabile "query" che contiene il codice SQL da eseguire. Il flag booleano **ack** viene ritornato dalla query, questo è **true** se viene ritornata almeno una riga come risultato, **false** in caso contrario o se viene sollevata un'eccezione. L'esecuzione della query si trova ovviamente all'interno di un blocco **try catch**, questo serve solamente per cambiare il valore del valore "ack" che si comporta come appena descritto. La variabile log contiene l'errore sollevato che viene poi inserito in un file specifico per gli errori del database, ovviamente scritto nel blocco catch.

3.6 Implementazione Grafici

In totale i grafici implementati sono **tredici**, di cui **cinque** per una descrizione dell'andamento generale di tutti gli iscritti all'esame, mentre i restanti **otto** sono specifici ai risultati dello studente scelto e fanno una comparazione dei risultati di quest'ultimo rispetto agli altri iscritti.

Avendo sviluppato la pagina in modo dinamico questa non viene mai ricaricata una volta che viene acceduta dall'utente grazie alle chiamate **AJAX**. Si è fatto quindi necessario l'utilizzo di due liste contenenti i riferimenti ad ogni grafico. I riferimenti ai grafici sono necessari dal momento in cui l'utente **cambia studente** e i dati all'interno dei grafici necessitano una variazione. Per poter aggiornare i grafici, senza ricaricamento della pagina, è obbligatorio chiamare la funzione destroy sul riferimento di ogni grafico che necessita un aggiornamento. Nella pagina dove vengono mostrati i grafici è anche presente un **bottone** che permette la stampa dei grafici visibili a schermo. La gestione dell'impaginazione in fase di stampa viene

fatta da codice **CSS**, inizialmente implementato usando la regole **media print**, che permette di applicare regole CSS solamente a specifici dispositivi, in questo caso solamente al documento prodotto utilizzando la funzione, del browser, stampa. Questo era stato l'approccio seguito inizialmente, in quanto il CSS era stato inserito all'interno di un foglio di stile separato rispetto a quello usato globalmente in LibreEOL. Successivamente il **CSS** è stato spostato nel foglio usato in tutto il sito, essendo questo l'approccio corretto, ma sono state riscontrati errori sull'impaginazione in fase di stampa; la **media query print** non veniva valutata. Il problema era dovuto al foglio di stile base di LibreEOL il quale è marcato come **media screen** al momento dell'import in HTML e quindi non valuta la media query print. Per risolvere questo problema è stato creato un foglio CSS con media print dove è stato inserito il codice CSS necessario per l'impaginazione di stampa.

Il documento con i grafici stampabile viene generato utilizzando la funzione JavaScript **print** che equivale al comando stampa del browser in uso. Ogni grafico ha un significato specifico ed è **interattivo**, infatti sono presenti animazioni **on hover** con etichette visibili solamente al passaggio del mouse. Vi sono anche animazioni in entrata dei grafici per esempio uno **slide in** o una rotazione nel grafico **pie** (a torta). Nella prossima sezione verrà descritto il significato di ogni grafico implementato.



Figura 3.4: Schermata Grafici Studente

3.7 Descrizione contenuto grafici specifici per ogni studente

Il **primo grafico** in figura è un grafico a torta che rappresenta il numero di domande facili/medie/difficili presenti all'interno del test. Nello specifico **ChartJs** chiama questo tipo di grafico **doughnut**, avendo la tipica forma della ciambella.

Il **secondo grafico** è un grafico a barre, rappresentante la percentuale di risposte giuste date dall'esaminato per ogni categoria di difficoltà presente nel test.

Il **terzo grafico** rappresentato è un **line chart**, dove viene rappresentata la media dei voti ottenuti nel test e il voto ottenuto dallo studente. Essendo rappresentata prima la media globale, il grafico sarà in **salita** se lo studente ha ottenuto un voto superiore alla media globale, mentre sarà in

discesa se avrà ottenuto un voto inferiore alla media globale.

Il **quarto grafico** è un **radar chart**, questo grafico rappresenta quante risposte giuste sono state date dall'esaminato per ogni genere di domanda. A differenza dei primi tre grafici queste categorie sono i tipi di domanda che sono: risposta aperta, vero/falso, sì/no, scelta multipla, scelta singola, risposta numerica. Nel grafico sono presenti, come etichette, le abbreviazioni usate nel sistema per ogni tipo di domanda e non vengono riportati i nomi interi per semplificare la lettura all'utente.

Il **quinto grafico** è sempre un **bar chart**, ma in questo caso orientato orizzontalmente e non verticalmente. Questo grafico confronta il **tempo impiegato in minuti** dall'esaminato per consegnare il test rispetto alla media globale, dove nella **barra rossa** viene rappresentato il tempo medio impiegato, mentre nella barra verde il tempo impiegato dallo studente.

Il **sesto e settimo grafico** sono entrambi **line chart**. Il primo rappresenta in verde il massimo dei punti ottenibili per ogni categoria (scelta multipla, vero/falso...) mentre in rosso il punteggio ottenuto dall'esaminato. Quindi se l'esaminato dovesse rispondere correttamente a tutte le domande, parte verde e parte rossa del grafico coinciderebbero.

Il **settimo grafico**, anch'esso un **line chart**, rappresenta nel suo punto a sinistra il minimo assoluto ottenuto nel test, al centro il punteggio ottenuto dallo studente e sulla destra il massimo assoluto ottenuto nel test.

L'**ottavo** e ultimo grafico del singolo studente, è un **bar chart** di un totale di sei barre. Questo grafico infatti è composto da tre coppie di barre, una per categoria di difficoltà. In ogni coppia vengono rappresentate il numero di domande giuste per difficoltà **facile**, **media**, **difficile**, nella

prima barra, mentre nella seconda è rappresentato il totale di domande presentate all'esaminato. Le barre viola rappresentano il totale delle domande presentate mentre la colonna gialla rappresenta il numero di risposte corrette.

3.8 Contenuto grafici globali

Scegliendo nel **drop down** degli studenti la voce "tutti" è possibile vedere tre grafici che rappresentano l'andamento di tutti gli studenti nell'esame.

Il **primo grafico** è un **line chart** che indica il voto preso da ogni studente. È possibile visualizzare precisamente in cifre il punteggio facendo hover con il cursore sopra ogni marker che mostrerà numero e nome/cognome dell'esaminato.

Il **secondo** grafico è un line chart che rappresenta due distribuzioni normali. Nella prima distribuzione in viola viene rappresentata la distribuzione ottenuta dai voti degli studenti, mentre in giallo abbiamo una distribuzione normale attesa la quale si spera di ottenere con i risultati degli studenti. La media della distribuzione normale attesa è di otto decimi o ventiquattro trentesimi.

Il **terzo** grafico fa uso di due tipi di barre una di colore verde e una di colore viola. La barra viola rappresenta il totale delle domande per categoria, mentre la barra verde mostra il numero di domande corrette per quella categoria. Le barre saranno quindi dodici essendo le categorie di domande sei.

Il **quarto** grafico è uno **scatter chart** che rappresenta i voti degli studenti rispetto alla media. In questo grafico viene infatti rappresentata la

media come una linea gialla parallela all'asse delle ascisse e i voti dei singoli studenti come indicatori circolari rossi. Se un indicatore sarà al di sopra della riga allora sarà intuitivamente superiore alla media, se invece sarà al di sotto della riga risulterà inferiore alla media.

Il **quinto grafico** è un **pie chart** che mostra nella sua zona gialla il punteggio ottenuto dagli studenti in totale , mentre nella sua zona verde il rimanente dei punti ottenibili, dando così uno spaccato della percentuale di risposte corrette nel globale.

3.9 Codice grafici

```
1 //-----Chart 7 info-----//
2 let dataChart7 = {
3   labels: [ttMiniumScore, ttStudent, ttMaximumScore],
4   datasets: [
5     {
6       label: ["Max Min"],
7       data: [
8         globalExamInfo["minScore"],
9         currentStudentTotalScore(),
10        globalExamInfo["maxScore"],
11      ],
12      backgroundColor: SOFT_YELLOW,
13      borderColor: HARD_YELLOW,
14      borderWidth: 1,
15    },
16  ],
17 };
```

Codice 3.6: Esempio di Data per grafico in chartJs

```
1 let optionsOnlyLabel = {
2   responsive: false,
3   plugins: {
4     title: {
5       display: true,
6       text: "label",
7     },
8   },
9   };
```

Codice 3.7: Esempio di options per grafico in chartJs

```
1 function createChart(ctx, typePar, dataPar, optionsPar) {  
2     return new Chart(ctx, {  
3         type: typePar,  
4         data: dataPar,  
5         options: optionsPar,  
6     });  
7 }
```

Codice 3.8: Funzione per creare grafico in chartsJs

Ogni grafico viene inserito all'interno dell'array precedentemente citato. Per creare un nuovo grafico è necessario istanziare un oggetto della classe **Chart**, la quale richiede come parametro una **canvas HTML** e un **JSON** che descriva le proprietà del grafico. Nella key "**type**" viene definito il tipo di grafico, chartJs ne mette a disposizione molti e molto personalizzabili [5], in questo caso si rappresenta un grafico di tipo line. Nella key "**data**" si inserisce un oggetto **JSON** che al suo interno contiene le etichette per ogni dataset, i dataset e varie impostazioni riguardante l'aspetto del grafico. La key "**dataset**" accetta come valore un array di **JSON**, in quanto questi possono essere più di uno. All'interno dei dataset verranno inseriti i dati ritornati dalle chiamate **AJAX** fatte nel codice. I valori dei colori, essendo spesso riutilizzati tra più grafici per dare senso di continuità, sono definiti a inizio file come variabili, utilizzando la convenzione delle maiuscole per indicare costanti. All'interno della key "**options**" viene passato un altro oggetto JSON. La sezione **options** serve per modificare molti comportamenti del grafico e permette di cambiare delle impostazioni generali a riguardo. In questo esempio option permette di personalizzare il titolo del grafico, una piccola scritta che compare al di sopra di esso.

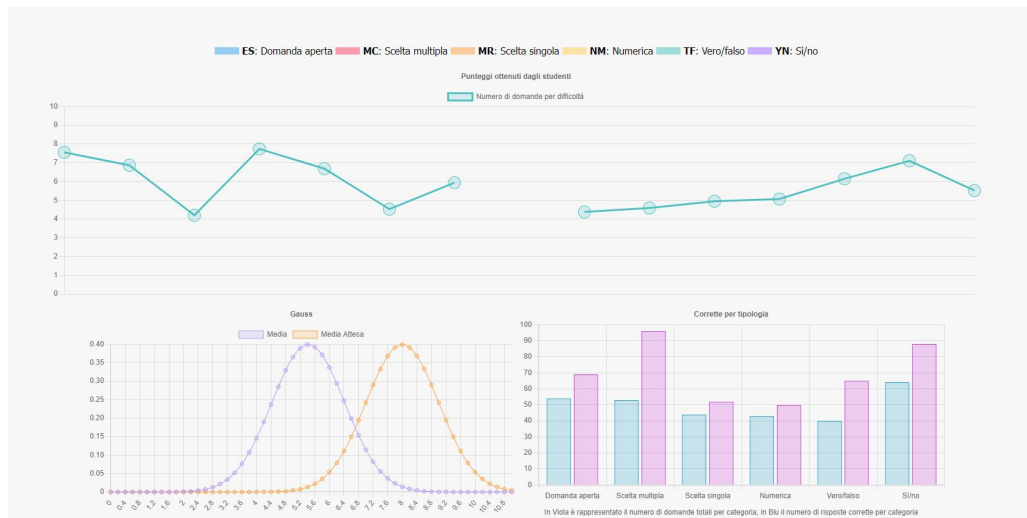


Figura 3.5: Schermata Grafici Globali

Nel progetto vengono creati tutti gli oggetti data, come mostrato nel listato di codice, come oggetto JSON e due oggetti options. Gli oggetti options sono comuni a tutti i grafici dei singoli studenti, tranne che per un grafico che avrà il suo specifico oggetto options. Per creare il grafico vero e proprio si chiama la funzione `createChart`, la quale ritorna un oggetto `chart`, prendendo come parametro i precedentemente citati oggetti `data` e `options`. Nel primo sviluppo si creava ogni grafico con una chiamata `new chart` inserendo direttamente gli specifici JSON all'interno di ognuna di esse. Questo approccio è risultato ripetitivo, poco leggibile e non pratico, quindi si è optato per questa soluzione che verrà approfondita in una sezione successiva riguarda la rifinitura del codice.

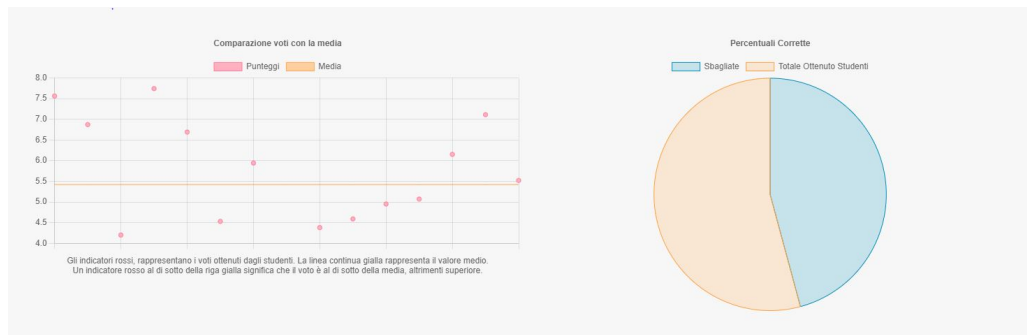


Figura 3.6: Schermata Grafici Globali

3.10 Traduzione

LibreEOL è un software che garantisce il suo utilizzo anche a persone straniere non in grado di parlare italiano, il che ha richiesto l'implementazione di traduzione dei grafici e dell'interfaccia utilizzata. Per effettuare questo tipo di traduzione è stato utilizzato il sistema già implementato all'interno del software che si basa sull'uso di file XML per la traduzione. Le traduzioni sono state fatte in due lingue, **italiano** e **inglese**. La scelta della lingua viene effettuata nella barra sottostante all'**header** del sito, qui l'utente è libero di scegliere la lingua da utilizzare selezionandola da un dropdown rappresentante un'icona della bandiera della lingua di scelta. La lingua scelta rimane in memoria anche dopo il logout dell'utente, essendo questa salvata all'interno di una apposita colonna all'interno della tabella users del database MySQL. Ogni parola che necessita di una traduzione viene inserita all'interno di un file **XML**, nel caso specifico del progetto i file XML saranno due uno per la lingua italiana e uno per la lingua inglese. Si è inizialmente creata la pagina non tradotta per testare l'interfaccia grafica e successivamente, ogni frase che necessitava la traduzione è stata inserita in

entrambi i file con il proprio ID. Infine il plain text viene sostituito con i relativi id del file XML che vengono compilati e la traduzione è funzionante. I file XML sono salvabili solamente mediante un account di **roulo admin**, che è in grado di raggiungere il pannello riservato a questa operazione in Sistema/Lingue.

```
1 <text id="ttGlobalChartScores">Scores obtained by students
   </text>
2 <text id="ttScores">Scores</text>
3 <text id="ttGlobalPieTitle">Percentage of corrected
   answers</text>
4 <text id="ttGlobalPieLabelCorrect">Total obtained by
   students</text>
```

Codice 3.9: Esempio Traduzione

Quando viene effettuata questa operazione vengono creati due file, uno in JavaScript e uno in PHP, in cui sono definite variabili di tipo stringa che hanno come nome gli id presenti nel file XML e il valore associato a quell'id. Con questi due file è dunque possibile richiamare l'id dato nel file XML all'interno dei file JavaScript o PHP e avere gli elementi necessari tradotti al cambio di lingua.

3.11 File JavaScript

La maggior parte del codice creato per il progetto si trova all'interno del file "CoachingReports.js". Questo file contiene tutta la gestione del front end del progetto. All'interno di questo file vengono istanziati i grafici come

descritto in precedenza e vi sono tutte le chiamate **AJAX** che forniscono i dati direttamente dal database.

```
1 //Returns the exam test Settings.
2 function getTestSettings() {
3     return getAjaxCall("index.php?page=exam/testsettings", "
4         post", false, {
5         idExam: globalExamInfo["idExam"],
6     });
7 }
```

Codice 3.10: Esempio funzione con chiamata AJAX

```
1 function getAjaxCall(urlPar, typePar, asyncPar, dataPar) {
2     let result = 0;
3     $.AJAX({
4         url: urlPar,
5         type: typePar,
6         async: asyncPar,
7         data: dataPar,
8         success: function (data) {
9             result = JSON.parse(data);
10        },
11        error: function (request, status, error) {
12            alert("jQuery AJAX request error:".error);
13            return error;
14        },
15    });
16    return result;
17 }
```

Codice 3.11: Codice funzione getAJaxCall

In figura viene mostrata un esempio di funzione con chiamata AJAX. La funzione incorpora al suo interno una chiamata alla funzione `getAjaxCall`, la quale restituisce l'oggetto di ritorno di una chiamata **AJAX** eseguita in **Jquery** alla quale vanno passate delle informazioni sotto forma di oggetto JSON. La funzione `getAjaxCall` è stata creata per ridurre il codice e renderlo più leggibile, inizialmente era stata ripetuta la sintassi della chiamata AJAX per ogni volta che queste si rendevano necessarie. Le parti fondamentali che definiscono una chiamata AJAX sin Jquery sono:

- **URL**, l'endpoint della chiamata, in questo URL viene definito il **controller** e l'**azione** che verranno eseguiti solamente nel caso in cui lo user che le chiama ha i diritti necessari per eseguirle.
- **type**, il tipo di **metodo http** che viene usato per la chiamata, spesso viene utilizzato il metodo **POST** in quanto è necessario passare parametri al database per eseguire query.
- **async**, indica se la chiamata deve essere **sincrona** o **asincrona**. Avendo avuto problemi utilizzando la chiamata in maniera asincrona si è preferito utilizzare la versione sincrona, rendendosi necessario l'utilizzo di una funzione con valore di ritorno.
- **data**, in questo **JSON** vengono passati i parametri POST necessari per l'interrogazione del database, nel caso specifico dell'esempio si passa l'**id** dell'**esame** preso in considerazione.
- **success**, in questa key viene definita una funzione da eseguire in caso di risposta da parte del server. Questa funzione viene eseguita anche

nel caso in cui la query restituisse zero righe come risultato. Il parametro della funzione "**data**" sono i dati ottenuti come **risultato** dalle interrogazioni.

- **error**, come per success si definisce una funzione che verrà però chiamata solamente nel momento in cui si dovesse presentare un errore nell'interrogazione al server.

Questa funzione presa in esempio, come mostra il nome, servirà per ottenere i **TestSettings** a livello frontend. La maggior parte delle chiamate AJAX sono state inserite all'interno di funzioni con prefisso get per aumentare la leggibilità e comprensione del codice, essendo la struttura delle chiamate comunque invadente a livello di visibilità di codice. Da parte server i dati vengono restituiti utilizzando la funzione **PHP** `json_encode` che trasforma i dati in convenzione JSON, ma che comunque vengono interpretati come stringa. Per convertire i dati da stringa ad oggetto json si fa uso del metodo **parse** della classe **JSON** che trasforma una stringa in oggetto JSON se questa ne rispetta la codifica.

All'interno del codice **JavaScript** si è fatto uso di un oggetto con visibilità globale chiamato **GlobalExamInfo**. `GlobalExamInfo` è un oggetto JSON che contiene tutte le informazioni riguardanti l'esame che non cambieranno fino al refresh o cambio della pagina, a prescindere dagli **input** dati dall'utente. Questo oggetto contiene quindi delle informazioni che ha senso ottenere a priori al caricamento della pagina senza dover interrogare nuovamente il server. I valori presenti nell'oggetto vengono inizializzati utilizzando un'apposita funzione chiamata **initExamObj**, all'interno della quale vengono chiamate tutte le funzioni che eseguono le interrogazio-

ni al db ritornando i valori che verranno assegnati ai campi dell'oggetto **GlobalExamInfo**.

```
1 let globalExamInfo = {
2   avg: 0,
3   avgDuration: 0,
4   totalEasy: 0,
5   totalMedium: 0,
6   totalHard: 0,
7   users: [],
8   arrayDuration: [],
9   testSettings: [],
10  minScore: 0,
11  maxScore: 0,
12  globalCatCorrect: [],
13  globalCatCount: [],
14  idExam: 0,
15 };
```

Codice 3.12: Esempio funzione con chiamata AJAX language

I campi dell'oggetto sono quasi tutti autoesplicativi e sono sia mostrati direttamente a schermo che utilizzati nelle varie computazioni dei grafici. Ad esempio il campo "**avg**" contiene la media totale in quell'esame e viene mostrato direttamente all'interno del **terzo grafico** dello specifico studente,. Se non si fosse fatto uso di un simile oggetto sarebbe stato necessario o l'uso di una singola variabile globale per valore dell'oggetto o di un'interrogazione al database ad ogni cambio di studente. La chiamata di funzione per l'inizializzazione dell'oggetto avviene al momento del caricamento della pagina.

```
1 $(document).ready(function () {
```

```
2   initExamObj();  
3 });
```

Codice 3.13: Inizializzazione oggetto globalExamInfo

Di seguito il corpo della funzione

```
1 function initExamObj() {  
2   globalExamInfo["idExam"] = $("#idExam").text();  
3   setGlobalExamInfoUsers();  
4   setGlobalExamInfoEMHTotalCount();  
5   setGlobalExamInfoArrayDuration();  
6   setGlobalExamInfoAvg();  
7   setGlobalExamInfoAvgTime();  
8   setGlobalExamInfoMaxMin();  
9   setGlobalExamInfoTestSettings();  
10  setGlobalExamInfoCategoriesNumbers();  
11 }
```

Codice 3.14: Esempio funzione con chiamata AJAX

Le funzioni precedute dal prefisso setGlobalExamInfo assegnano il **valore** alla **key** dell'oggetto all'interno del success delle chiamate AJAX che si trovano al loro interno, è possibile effettuare questo assegnamento dal momento che l'oggetto globalExamInfo ha visibilità globale.

3.12 CSS

Il codice CSS inizialmente è stato scritto su un foglio a se stante per effettuare il testing, successivamente è stato incluso nel foglio di stile globale di LibreEOL, locato nella cartella "themes/default". Il codice viene scritto in fondo, sovrascrivendo quindi possibili conflitti o regole già scritte in

precedenza. La funzione principale del **CSS** implementato è quella relativa allo stilare la pagina di stampa dei grafici. Per poter stilare questa pagina inizialmente si è usata la **media query print** che applica il codice solamente nel momento in cui si cerca di stampare la pagina corrente, ma per i problemi citati già nelle precedenti sezioni la soluzione finale prevede la creazione di un foglio separato con media print, all'interno del quale ovviamente non serve chiamare la media query print. Questo codice nel pratico allinea i grafici in modo gradevole per l'utente e cerca di suddividerne quattro per pagina, includendo il **nome** dello **studente**, il **nome** del **esame** che viene stampato e la data in cui questo è stato svolto. Il codice CSS inoltre modifica il valore di **display** di molti elementi. Il valore viene modificato usando "display:none", rendendo così **invisibili** molti elementi che non vanno stampati poiché renderebbero la pagina illeggibile o sarebbero solamente di intralcio alla lettura. Ad esempio viene tolto il **drop down** degli studenti, poiché è un elemento utile solamente in un ambiente web e viene sostituito con il suo **valore** evitando così il riquadro e la tipica forma dell'elemento. Per mostrare questo contenuto è presente una **div** all'interno della quale viene inserito il valore e resa visibile con il processo contrario a quello che accade al drop down e cioè settare il valore di display a "**inline**" dal valore di partenza "**none**" quando si è nella pagina web. Questo avviene poiché l'informazione del nome dello studente nella pagina web si trova nel drop down e risulterebbe solamente ripetitiva se mostrata anche nella div, fondamentale però per mostrare l'informazione sotto forma di testo piuttosto che con una grafica da drop down nel foglio stampato. Ovviamente anche i tasti vengono resi invisibili al momento della stampa poiché elemen-

ti prettamente web e non necessari nel foglio stampato in cui si perde la loro utilità. Vengono resi **invisibili** inoltre anche barra di azioni, header e footer della pagina. Automaticamente il browser inserisce l'host e il sito dal quale viene stampata la pagina, anche queste informazioni sono rimosse rimuovendo il margine della pagina.

3.13 Migrazione codice da locale a web server del dipartimento

Come già detto in precedenza, l'intero progetto è stato sviluppato in **locale**. Lo sviluppo in locale è stato eseguito mediante l'utilizzo del software **XAMPP** su Linux Mint 21 "Vanessa" come sistema operativo della macchina. XAMPP è un software **open source** che permette lo sviluppo in locale di web application utilizzando lo stack di sviluppo dei classici server apache. Visto che molti webserver live utilizzano lo stesso stack utilizzato da XAMPP risulta anche spesso molto semplice la transizione da locale a live server. Lo stack di cui si parla implementa Apache, MariaDB, PHP e Perl, alcuni dei quali sono componenti fondamentali già descritti per lo sviluppo del progetto.

La scelta dello sviluppo in locale nella prima fase del progetto è dovuta alla rapidità dello sviluppo e la maggiore libertà nella gestione dell'intero sistema, di fatti, come detto a inizio sezione, lo sviluppo in locale ha interessato solamente la prima parte del progetto che è stato migrato poi nella sua fase conclusiva.

Tutti i file del progetto sono stati compressi e inseriti all'interno del web

server del dipartimento e stesso procedimento è stato effettuato per il database utilizzato in locale, del quale è stato creato un file **gz** che poi è stato caricato all'interno del database del web server live. La migrazione del database non era fondamentale ai fini del funzionamento del progetto, in quanto la sua struttura è rimasta invariata e non sono state né create né modificate tabelle; la scelta della migrazione è stata fatta per mantenere i dati di test utilizzati durante lo sviluppo per continuare a testare l'implementazione sia per avere dei dati per fare dimostrazioni del risultato finale.

Per comunicare e effettuare modifiche al live server, più protocolli sono coinvolti per la riuscita delle anche più semplici modifiche da effettuare. Il primo protocollo involuto nella comunicazione è **SSH**, acronimo di secure shell, è un protocollo di comunicazione rete che permette a due computer di scambiare informazioni in modo sicuro essendo la comunicazione fra le due macchine criptata. Viene spesso utilizzato per effettuare login in remoto o eseguire comandi da terminale in remoto su altre macchine. **SSH** nasce su ambienti **unix** e ha come obiettivo principale la sostituzione di **Telnet** e altri protocolli non sicuri come **rsh** e **rlogin**.

La connessione, mediante SSH appunto, nel caso del progetto è stata effettuata da una macchina **windows**. Come primo passaggio è stato creato un **alias SSH** per semplificare la connessione via linea di comando, questo perché ad ogni connessione occorre specificare su riga di comando **username**, **porta**, **remote host** e **file di chiave**. Per evitare questa lunga e tediosa procedura, che può anche portare ad errori nello scrivere il comando, si crea quindi l'alias andando a modificare il file config nella cartella `/user/.ssh` come segue:

```
1 Host brufaniebaglioni
2     HostName woodie.unipg.it
3     User brufaniebaglioni
4     IdentityFile pathToKey
```

Codice 3.15: Esempio funzione con chiamata AJAX language

All'interno di questo file vengono specificate tutte le informazioni necessarie all'esecuzione del comando già prima citato. Dopo la creazione di questo alias sarà sufficiente scrivere a riga di comando "ssh alias" e la connessione inizierà.

Nel caso la macchina non avesse ancora una coppia di **chiavi SSH**, di conseguenza mancherà anche la cartella .ssh all'interno della quale inserire l'alias sopra descritto, occorrerà chiamare il comando "ssh-keygen" che creerà una coppia di chiavi SSH pubblica e privata con la conseguente cartella ".ssh".

Per la modifica dei file in remoto è stato utilizzato l'IDE "phpStorm" di JetBrains. Mediante l'uso di questo IDE è possibile modificare i file all'interno di esso avendo tutti i vantaggi dell'ambiente di sviluppo. Per effettuare queste modifiche si esegue una connessione mediante il protocollo SFTP. Dopo aver effettuato la connessione è possibile modificare i file direttamente dall'IDE ed effettuare l'upload al web server in pochi passaggi, avendo le modifiche in tempo reale.

All'interno del sistema è stato creato un **utente non root** per rendere possibile la modifica dei file. I file del progetto infatti sono stati forniti di **permessi 775** all'utente dedicato. Grazie a questo livello di permessi l'utente assegnato è in grado di leggere, scrivere ed eseguire i suddetti file senza rischiare di interferire con il sistema del web server.

Per visualizzare la struttura file con la classica interfaccia windows è stato utilizzato il software consigliatomi "RaiDrive" che permette una gestione dei file, sempre mediante il protocollo **SFTP**, più semplice ed immediata rispetto alla CLI.

La gestione del database è rimasta al software **phpMyAdmin** che, rispetto al locale, ha ovviamente cambiato solo le credenziali di accesso e l'URL per accedervi.

Il progetto è stato quindi messo online all'indirizzo "dev2.libreeol.org".

Capitolo 4

Processo di sviluppo

In questo capitolo vengono descritte delle considerazioni fatte durante lo svolgimento del progetto e accorgimenti che hanno portato al **refactoring** del codice per migliorarne le prestazioni, leggibilità e ridurre la lunghezza. Molte delle modifiche applicate sono state dettate da canoni espressi nel libro "Clean Code" di *Robert Cecil Martin*[8].

4.1 Prima suddivisione del codice

Il primo sviluppo del codice prevedeva funzioni di notevole lunghezza e poco chiare alla prima lettura.

Per ovviare al problema il codice è stato diviso in molte funzioni che hanno come prerogativa l'essere il più corte e comprensibili possibile. La struttura del codice dopo il primo refactoring effettuato ha assunto una sola chiamata nella funzione **Ready** che serve per inizializzare l'oggetto global GlobalExamInfo.

Le funzioni sono state poi divise in gruppi logici.

Il primo gruppo è formato da tutte le funzioni chiamate per inizializzare l'oggetto **GlobalExamInfo**.

Il secondo gruppo rappresenta le funzioni get, tutte le funzioni che iniziano con il prefisso get, che chiamano sempre una **AJAX** che restituisce un certo tipo di dato.

Sono presenti poi due macro funzioni più lunghe delle altre adibite l'una alla istanziazione dei grafici specifici degli studenti e l'altra all'istanziazione dei grafici per tutti gli studenti iscritti.

4.2 Secondo refactoring

Dopo aver suddiviso il codice in molte funzioni è stata notata la presenza di una ripetizione smisurata delle chiamate AJAX che eseguivano tutte lo stesso compito e cioè il semplice ritorno dei dati richiesti e l'assegnazione ad una variabile.

Per seguire una politica detta in gergo "DRY" (don't repeat yourself) , cioè evitare la ripetizione di codice non necessaria, è stata creata la funzione `getAjaxCall`.

```
1 function getAjaxCall(urlPar, typePar, asyncPar, dataPar) {  
2   let result = 0;  
3   $.ajax({  
4     url: urlPar,  
5     type: typePar,  
6     async: asyncPar,  
7     data: dataPar,  
8     success: function (data) {  
9       result = JSON.parse(data);
```

```

10     },
11     error: function (request, status, error) {
12         alert("jQuery AJAX request error:".error);
13     },
14 });
15 return result;
16 }
17 }
```

Codice 4.1: Esempio funzione con chiamata ajax

Questa funzione riceve l'**endpoint**, il **tipo** di chiamata, un **booleano** per la scelta se la chiamata debba essere **sincrona** o **asincrona**, i **parametri POST** e ritorna i dati in formato **JSON**. Grazie a questa funzione è stato possibile ridurre il codice di qualche centinaio di righe e aumentarne la leggibilità in quanto una chiamata ajax di questo tipo si riduce a una singola riga, mentre in precedenza venivano occupate almeno una ventina di righe per chiamata.

Stesso principio è stato applicato per la istanziazione dei grafici, questi venivano generati uno ad uno facendo una chiamata al costruttore della classe Chart della libreria ChartJs. Successivamente per correggere questa ripetizione non necessaria sono state divise le componenti di ogni grafico. Ogni grafico è stato decomposto in due oggetti **JSON** fondamentali, l'oggetto **data** e l'oggetto **options**. Quindi per ogni grafico avremo ora solamente due JSON che vengono passati alla funzione **createChart** che insieme al context (cioè la canvas HTML) e il tipo di grafico questa creerà il grafico. Per ridurre ulteriormente le righe di codice e evitare ripetizioni i tipi di grafici, i riferimenti agli oggetti options e data sono stati raggruppati in due

liste che vengono passate alla chiamata di funzione in un ciclo for e infine assegnate all'array dei riferimenti dei grafici.

```
1  for (let i = 0; i < arrayOfSingleStudentCharts.length; i
    ++ ) {
2      arrayOfSingleStudentCharts[i] = createChart(
3          $("#chart" + (i + 1)),
4          chartType[i],
5          dataChart[i],
6          optionsChart[i]
7      );
```

Codice 4.2: Creazione di tutti i grafici studenti

Con questa suddivisione è possibile raggruppare tutti i dati dei grafici e le loro opzioni.

Una ulteriore riduzione del codice è avvenuta dopo aver notato che tutti i grafici tranne uno facevano uso dello stesso oggetto options. Sono stati creati quindi due oggetti **options**, uno per un singolo grafico e l'altro per i rimanenti. Successivamente con un **ciclo for** l'oggetto options viene modificato con le label specifiche del grafico che viene istanziato al momento. Queste label utilizzate per la creazione dell'oggetto options sono mantenute all'interno di una lista che viene scansionata al momento della creazione/modifica di un grafico.

```
1  for (let i = 0; i < arrayOfSingleStudentCharts.length; i
    ++ ) {
2      let optionsOnlyLabel = {
3          responsive: false,
4          plugins: {
5              title: {
```

```
6         display: true,
7         text: "label",
8     },
9 },
10 };
11 optionsOnlyLabel["plugins"]["title"]["text"] = labels[i];
```

Codice 4.3: Ciclo for per utilizzo dell'oggetto options

Ovviamente lo stesso ragionamento è stato applicato anche ai **grafici globali**, con la differenza che essi avevano un oggetto options diverso rispetto a quello dei singoli studenti.

Conclusioni

Il progetto è risultato molto stimolante e formativo in quanto mi ha permesso di lavorare su un vero software e implementare all'interno di un ambiente non completamente sviluppato da me.

Questo tipo di esperienza è risultata formativa in quanto nella carriera di software development raramente si scrive codice da zero e piuttosto si investe la maggior parte del tempo nella manutenzione di codice esistente o implementazione di nuove funzionalità all'interno di strutture già esistenti, come nel caso del progetto. Per la prima volta mi sono interfacciato con un software che faceva uso del pattern strutturale MVC, il quale è risultato molto interessante e che ho approfondito anche nel privato.

Molto formativa è stata anche la possibilità di leggere codice molto più robusto rispetto a quello che solitamente scrivo e comprendere delle pratiche che sicuramente utilizzerò ancora nei miei progetti. L'intervento dei relatori nella revisione del codice scritto e i consigli per il refactoring e la migliaoria del codice sono stati determinanti, mettendo anche alla luce lacune e errori che da ora in poi non verranno più ripetuti. Gli obiettivi del progetto sono stati raggiunti permettendomi di ampliare la mia conoscenza riguardo alla programmazione web, campo che mi interessa molto, e favorendo lo sviluppo della mia carriera futura.

Bibliografia

- [1] **Bernard Marr**, *Key Performance Indicators (KPI): The 75 measures every manager needs to know*, Settembre 2012
- [2] **John Deacon**, *Model-View-Controller (MVC) Architecture*, August 1995, revisionato August 2000, Aprile 2005 and Maggio 2009.
- [3] **John Deacon**, *Model-View-Controller (MVC) Architecture*, nello specifico pagina 2 sezione dedicata alla view.
- [4] **Jeremy Keith, Jeffrey Sambells**, *DOM Scripting: Web Design with JavaScript and the Document Object Model*, 7 Agosto 2011, nello specifico pagina 100.
- [5] , **Helder da Rocha**, *Learn Chart.js: Create interactive visualizations for the Web with Chart.js 2*, 28 Febbraio 2019.
- [6] **Justin Clarke**, *SQL Injection Attacks and Defense*, 2012.
- [7] **Tiffany B Brown**, *CSS Master*, Agosto 15, 2021, capitolo "Selectors".
- [8] **Robert Cecil Martin**, *Clean Code*, Agosto 2008.

Appendice A

Codice